Security Vulnerability Report

Executive Summary

Scan Overview:
• Scan ID: 019dd797-6a79-469c-8324-015ed01f1a01
• Date: 2025-11-09T06:14:07.923Z
• Files Scanned: 1
• Total Vulnerabilities: 6

Severity Breakdown:
• Critical: 0
• High: 2
• Medium: 4
• Low: 0

Vulnerability Summary

File	Line	Type	Severity	CWE
files-1762668847942-409047794.py	25	sqlalchemy-execute-raw-query	High	CWE-CWE-89: Improper Neutralization of Special Elements used in an SQ
files-1762668847942-409047794.py	38	subprocess-shell-true	High	CWE-CWE-78: Improper Neutralization of Special Elements used in an OS Com
files-1762668847942-409047794.py	25	formatted-sql-query	Medium	CWE-CWE-89: Improper Neutralization of Special Elements used in an SQ
files-1762668847942-409047794.py	46	eval-detected	Medium	CWE-CWE-95: Improper Neutralization of Directives in Dynamically Evalu
files-1762668847942-409047794.py	55	avoid-pickle	Medium	CWE-CWE-502: Deserialization of Untrusted Da
files-1762668847942-409047794.py	81	md5-used-as-password	Medium	CWE-CWE-327: Use of a Broken or Risky Cryptographic

Detailed Vulnerability Findings

Finding 1: sqlalchemy-execute-raw-query

File: files-1762668847942-409047794.py
Line: 25
Severity: High
CWE ID: CWE-CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Detection Tool: Semgrep
Confidence: 85%

Description:
Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option. Recommendation: Use parameterized queries (prepared statements) instead of string concatenation. Example: ```python cursor.execute("SELECT * FROM users WHERE id = %s", [user_id]) ```

Code Snippet:

```
23:     # UNSAFE: string formatting of user input
24:     query = "SELECT id, username FROM users WHERE username LIKE '%{}%';".format(q)
25:     cur = conn.execute(query)
26:     rows = cur.fetchall()
27:     conn.close()
```

Finding 2: subprocess-shell-true

File: files-1762668847942-409047794.py
Line: 38
Severity: High
CWE ID: CWE-CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
Detection Tool: Semgrep
Confidence: 85%

Description:
Found 'subprocess' function 'call' with 'shell=True'. This is dangerous because this call will spawn the command using a shell process. Doing so propagates current shell settings and variables, which makes it much easier for a malicious actor to execute commands. Use 'shell=False' instead. Recommendation: Review the code for security implications and consider using secure coding practices appropriate for your use case.

Code Snippet:

```
36:     cmd = "ping -c 1 " + target
37:     # UNSAFE: shell=True with concatenated user input
38:     subprocess.call(cmd, shell=True)
39:
40: # ===== Eval (RCE) =====
```

Finding 3: formatted-sql-query

File: files-1762668847942-409047794.py
Line: 25
Severity: Medium
CWE ID: CWE-CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Detection Tool: Semgrep
Confidence: 85%

Description:
Detected possible formatted SQL query. Use parameterized queries instead. Recommendation: Use parameterized queries (prepared statements) instead of string concatenation. Example: ```python cursor.execute("SELECT * FROM users WHERE id = %s", [user_id]) ```

Code Snippet:

```
23:     # UNSAFE: string formatting of user input
24:     query = "SELECT id, username FROM users WHERE username LIKE '%{}%';".format(q)
25:     cur = conn.execute(query)
26:     rows = cur.fetchall()
27:     conn.close()
```

Finding 4: eval-detected

File: files-1762668847942-409047794.py
Line: 46
Severity: Medium
CWE ID: CWE-CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')
Detection Tool: Semgrep
Confidence: 85%

Description:
Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this content can be input from outside the program, this may be a code injection vulnerability. Ensure evaluated content is not definable by external sources. Recommendation: Review the code for security implications and consider using secure coding practices appropriate for your use case.

Code Snippet:

```
44:     """
45:     # UNSAFE: eval on raw user input
46:     return eval(user_expr)
47:
48: # ===== Insecure Deserialization =====
```

Finding 5: avoid-pickle

File: files-1762668847942-409047794.py
Line: 55
Severity: Medium
CWE ID: CWE-CWE-502: Deserialization of Untrusted Data
Detection Tool: Semgrep
Confidence: 85%

Description:
Avoid using `pickle`, which is known to lead to code execution vulnerabilities. When unpickling, the serialized data could be manipulated to run arbitrary code. Instead, consider serializing the relevant data as JSON or a similar text-based serialization format. Recommendation: Review the code for security implications and consider using secure coding practices appropriate for your use case.

Code Snippet:

```
53:     """
54:     # UNSAFE: pickle can execute arbitrary code during unpickling
55:     obj = pickle.loads(data_bytes)
56:     return obj
57:
```

Finding 6: md5-used-as-password

File: files-1762668847942-409047794.py
Line: 81
Severity: Medium
CWE ID: CWE-CWE-327: Use of a Broken or Risky Cryptographic Algorithm
Detection Tool: Semgrep
Confidence: 85%

Description:
It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by an attacker in a short amount of time. Use a suitable password hashing function such as scrypt. You can use `hashlib.scrypt`. Recommendation: Move sensitive data to environment variables or a secure configuration management system: ```python import os password = os.environ.get("DB_PASSWORD") ```

Code Snippet:

```
79:     Vulnerable: using MD5 for password hashing (cryptographically broken).
80:     """
81:     return hashlib.md5(password.encode()).hexdigest()
82:
83: # ===== Demo runner (for local testing only) =====
```

AI-Generated Recommendations

• Implement parameterized queries for all database operations to prevent SQL injection
• Use environment variables for storing sensitive configuration data
• Implement proper input validation and output encoding
• Use strong cryptographic functions (bcrypt, scrypt, or Argon2) for password hashing
• Regular security code reviews and automated scanning
• Keep dependencies updated and monitor for known vulnerabilities